

 **Shared by lokesh.** This is a copy of a chat between Claude and lokesh. Content may include unverified or unsafe content that does not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

Report

I'm doing a lab where I have to fix a broken Pac-Man game in Pygame and add features. I've attached game.py and the README. Can you first explain how the code is structured, then help me with Task 1? The README says eating a power pellet does nothing beyond 10 points and frightened mode never triggers. Can you find the bug in eat()?

I'll take a look at the files first.

Looked for files, ran a command

The files didn't come through. Your uploads folder is empty, so I don't have `game.py` or the README.

You can get them to me in either of two ways:

- **Attach them again** to your next message (drag both into the chat).
- **Tell me where they are on your computer.** This chat is linked to your computer, but no folders are shared with it yet. Click the "+" beside the message box, choose "Add folder", and pick the lab folder. I'll read the files from there.

Once I have the code, I'll explain how it's structured, then go through `eat()` to find the power-pellet bug.

While you're getting the files, here are the bugs that most often cause this exact symptom in Pac-Man labs. You get the 10 points, but frightened mode never starts:



1. **The tile is cleared before it's checked.** The code sets the tile to empty first, then checks `if tile == POWER`. That check can never be true.
2. **The wrong constant.** The power branch compares against `PELLET` instead of `POWER_PELLET`, or the two constants are equal. Then the regular-pellet branch always catches it first.
3. **if / elif order.** A broad check like `if tile in (PELLET, POWER):` scores 10 and returns before the power branch runs.
4. **The flag is set on a copy.** The code sets something like `frightened = True` on a local variable or the wrong object, instead of on each ghost or on the game state.
5. **The timer is set but never counted down.** Or it's set to 0, or `update()` resets it every frame, so frightened mode ends the moment it starts.

When I see your `eat()` I'll tell you which one it is, or whether it's something else.

Files hidden
when shared



Ran a command, shared files

How the code is structured

It's all one file, in four layers:

1. Constants and the maze (lines 4–29)

- `MAZE` is a list of strings, one character per tile: `#` is a wall, `.` is a pellet, `o` is a power pellet, and a space is empty (the ghost house).



- The grid uses `(row, col)` coordinates everywhere, and `TILE = 28` pixels per tile.
- Also defined here: `DIRS` and `KEY_DIRS` (movement vectors), the ghost house cells and exit, the player's start, `FRIGHT_SECONDS = 3.0`, and the step intervals. The player moves every 0.14 s and ghosts every 0.17 s.

2. Hook functions you'll fill in (lines 32-44)

- `ghost_color`, `on_pellet_eaten` and `bonus_life_threshold` all `pass`, so they return `None`.
- The game is already wired to call them, which is why tasks 2-4 only ask you to fill in these bodies.

3. `Ghost` class (lines 60-114)

- `target()` picks the tile each ghost aims for:
 - An eaten ghost goes home.
 - A ghost inside the house heads for the exit.
 - In scatter phase, it goes to its corner.
 - In chase phase, each ghost has its own rule: Blinky targets the player, Pinky 4 tiles ahead of the player, Inky uses a vector from Blinky, and Clyde chases until he's within 8 tiles, then retreats to his corner.
- `options()` lists the legal moves. A ghost can't go into walls, can't re-enter the house unless it's eaten, and can't reverse unless it has no other choice.
- `step()` moves one tile. A frightened ghost picks a random direction; otherwise it picks the move that gets closest to its target.
- `reverse()` flips its direction. This is used when a power pellet is eaten.

4. `Game` class (lines 117-248)



- **State:** `reset()` sets up a new game and `respawn()` handles losing a life. `self.pellets` is a set of the remaining pellet positions.
- `update(dt)` runs every frame:
 1. Advances the clock and counts down `fright_left`.
 2. Checks for a bonus life.
 3. Moves the player whenever its 0.14 s interval has passed.
 4. Moves the ghosts whenever their 0.17 s interval has passed. Frightened ghosts skip every other step, so they're half speed, and each ghost waits for its `release` time before leaving.
 5. Checks for collisions.
- `chasing()` alternates the ghosts' mode: 7 s scatter, then 20 s chase, in a 27 s cycle.
- `move_player()` → `eat()`: whenever the player moves onto a tile, `eat()` runs on it.
- `check_collisions()`: if a frightened ghost touches the player, the ghost is eaten and you get +200. Otherwise you lose a life.
- `draw()`: draws the maze, Pac-Man, the ghosts (blue, flashing white in the last second of frightened mode) and the score bar.

`main()` is the standard Pygame loop: handle events, call `update`, call `draw`, then flip the display.

Task 1: the bug in `eat()`

Line 160 compares the tile against an uppercase `"O"`:

```
python
if MAZE[cell[0]][cell[1]] == "O":
```

But the maze marks power pellets with a lowercase `"o"`, for example

`"#o.....#.....o#"`. That condition is never true. So eating a power pellet only runs



the regular `self.score += 10`, and the frightened-mode block never executes.

Notice that `reset()` (line 128) and `draw()` (line 220) both correctly use lowercase `o`. That's why power pellets still appear and can be eaten; only `eat()` has the wrong letter.

The fix:

```
python
if MAZE[cell[0]][cell[1]] == "o":
```

I tested it by eating the top-left power pellet in both versions:

	Score	fright_left	Ghosts reversed?
Original	10	0.0	No
Fixed	50 (10 + 40)	3.0	Yes

The fixed file is `task1_game.py`. Your commit script copies it from `Lab-4\code_by_task\`, so save it there.

For your "before" video, eat a corner power pellet and run into a ghost: you'll lose a life. In the "after" video, the ghosts turn blue, run the other way, and you get +200 when you catch one.

That fixed it, ghosts turn blue and reverse now. Next is Task 2, implementing `ghost_color(name, mode)`. I want each ghost to have its own colour in frightened mode instead of all of them being the same blue, so I can still tell them apart. Return None for normal and eaten modes so they keep their default look. Give me only the function.



```
python

def ghost_color(name, mode):
    """Return an (r, g, b) colour override for a ghost, or None to keep the default.
    # Note: returning a colour here also replaces the white flash in the last second
    if mode != "frightened":
        return None
    frightened_colors = {
        "blinky": (150, 60, 200),    # purple
        "pinky": (255, 180, 230),    # pale pink
        "inky": (40, 140, 255),      # sky blue
        "clyde": (120, 200, 90),     # green
    }
    return frightened_colors.get(name)
```

Now Task 3, `on_pellet_eaten(score, pellets_left)`. I want a bonus cherry to appear below the ghost house at (7, 10) when `pellets_left` reaches 100 and again at 50. It should stay for about 9 seconds and give 100 points when Pac-Man walks over it. The problem is the return value of this function is ignored and it can't access the Game object, so how do I get the fruit into the game? Show me the changes needed in Game (reset, update, move_player, draw) as well.

Ran 3 commands, read a file, shared files

The hook can't touch `Game` and its return value is ignored, but it can write to something at module level that `Game` also reads. So `on_pellet_eaten` drops a request into a shared list `fruit_requests`, and the `Game` turns each request into a cherry right after the move that



ate the pellet. The hook only says *when* a fruit should appear, and `Game` keeps the fruit's timer, scoring and drawing.

One thing to know: `(7, 10)` is directly *above* the ghost house (the house is rows 8-9), on the tile where ghosts come out. I kept that tile; change `FRUIT_CELL` if you meant somewhere else.

1. Constants and shared list (put these after `PLAYER_STEP, GHOST_STEP = ...`)

python

```
FRUIT_CELL = (7, 10)
FRUIT_SECONDS = 9.0
FRUIT_POINTS = 100
FRUIT_TRIGGERS = (100, 50)
fruit_requests = [] # on_pellet_eaten() appends here; Game picks it up
```

2. The hook

python

```
def on_pellet_eaten(score, pellets_left):
    """Called after every pellet is eaten; add sound, flashes, or bonus fruit here."""
    if pellets_left in FRUIT_TRIGGERS:
        fruit_requests.append(FRUIT_CELL)
```

3. `reset()`: add these two lines before the `for ghost in self.ghosts:` loop

python

```
self.fruit_left = 0.0
fruit_requests.clear()
```



4. `move_player()`: call a new method after `eat()`, and add that method to `Game`

python

```
        self.eat(tuple(cell))
        self.check_fruit()
        return

def check_fruit(self):
    # Collect first, then spawn, so a cherry that appears under Pac-Man isn't ea
    if self.fruit_left > 0 and tuple(self.player) == FRUIT_CELL:
        self.fruit_left = 0.0
        self.score += FRUIT_POINTS
    if fruit_requests:
        fruit_requests.clear()
        self.fruit_left = FRUIT_SECONDS
```

5. `update()`: count the timer down, next to the `fright_left` line

python

```
self.fright_left = max(0.0, self.fright_left - dt)
self.fruit_left = max(0.0, self.fruit_left - dt)
```

6. `draw()`: draw the cherry just before the `px, py = ...` line, so Pac-Man and the ghosts are drawn on top of it

python



```

if self.fruit_left > 0:
    fx, fy = FRUIT_CELL[1] * TILE + TILE // 2, FRUIT_CELL[0] * TILE + TILE /
    pygame.draw.line(screen, (60, 180, 60), (fx - 5, fy + 4), (fx + 2, fy -
    pygame.draw.line(screen, (60, 180, 60), (fx + 5, fy + 4), (fx + 2, fy -
    pygame.draw.circle(screen, (220, 20, 40), (fx - 5, fy + 4), 5)
    pygame.draw.circle(screen, (220, 20, 40), (fx + 5, fy + 4), 5)

```

A few details to check:

- **The order inside `check_fruit()`.** The pellet at `(7, 10)` could be the one that brings the count down to 100. If the code spawned first and checked for collection second, Pac-Man would collect the cherry the instant it appeared under him. Checking first means he has to step off and come back for it.
- **Both triggers fire.** The maze has 130 pellets, so the count hits 100 after 30 are eaten and 50 after 80. `pellets_left` drops by exactly one per pellet, so `in (100, 50)` fires exactly once each.
- **Reset is clean.** `reset()` clears the shared list too, so a cherry request can't carry over into the next game.

I checked it headlessly. The cherry appears at 100 and 50 pellets left, walking over it adds 100 on top of the pellet's 10, and it disappears after 9 seconds. My test screenshot also showed the Task 2 frightened colours working.

`task3_game.py` has Tasks 1–3 applied, ready for `Lab-4\code_by_task\`.



Last one, Task 4, `bonus_life_threshold()`. The README suggests returning 10000, but I counted the maze and there are only about 130 pellets, so the max score is around 1500. 10000 will never be reached, right? What value makes sense? Also, can I show a small "EXTRA LIFE!" message on the HUD when it happens?

Ran 2 commands, read 2 files, shared files

You're right, 10000 is out of reach in this maze. Your count is correct: 126 regular pellets and 4 power pellets. Here's every source of points:

Source	Points
126 pellets $\times$ 10	1260
4 power pellets $\times$ 50 (10 + 40)	200
2 cherries $\times$ 100	200
Ghosts	200 each, but only during 3-second frightened windows

Clearing the board earns 1460 points, or 1660 if you also get both cherries. Even catching every ghost after every power pellet ($4 \times 4 \times 200 = 3200$) is far from 10000. The README's number was probably copied from the real arcade game, which has a much bigger maze.

I'd use 1000. You reach it about two-thirds of the way through the board, or sooner if you catch some ghosts. Every winning game passes it, so the feature will actually show up in your "after" video. A good run with some ghost-eating might pass 2000 and get a second life. A lower value like 500 gives out lives too easily: you'd get two or three just from pellets.

```
python
```



```
def bonus_life_threshold():
    """Return a score value at which the player earns an extra life, or None to disa
    return 1000
```

The "EXTRA LIFE!" message needs a small timer in `Game`, set in the bonus-life block that's already there. It touches three methods.

`reset()`: add this next to the other timers

python

```
self.extra_life_left = 0.0
```

`update()`: set the timer when a life is awarded, and count it down each frame

python

```
self.fruit_left = max(0.0, self.fruit_left - dt)
self.extra_life_left = max(0.0, self.extra_life_left - dt)
threshold = bonus_life_threshold()
if threshold and self.score // threshold > self.bonus_awarded:
    self.bonus_awarded = self.score // threshold
    self.lives += 1
    self.extra_life_left = 2.0
```

`draw()`: add this right after the HUD `screen.blit(hud, ...)` line

python



```
if self.extra_life_left > 0 and int(self.extra_life_left * 4) % 2 == 0:  
    bonus = font.render("EXTRA LIFE!", True, (120, 255, 120))  
    screen.blit(bonus, bonus.get_rect(topright=(W - 8, ROWS * TILE + 6)))
```

The message appears in green at the right end of the HUD bar and blinks for 2 seconds. I tested it:

- At a score of 990 you still have 3 lives.
- At 1000 you get a 4th life and the message appears.
- The next frame doesn't award another life.
- At 2000 you get a 5th.

`task4_game.py` has all four tasks applied, so it can go straight into `Lab-4\code_by_task\` for your commit script.

Can you give me the complete final game.py with all 4 tasks done, so I can check it against mine?

3 minutes ago 

Ran 4 commands, shared files

I've sent `game.py`, the complete final version with all four tasks done. It compiles cleanly, and each task behaved as expected in my headless tests. To compare it with yours, `changes.diff` lists only what changed from the original file, about 50 lines.

I also fixed a problem in the `task3_game.py` and `task4_game.py` I sent earlier. Your original file uses Windows line endings (CRLF), but those two had been saved with Unix line endings. Git would have counted every line as changed, so your Task 3 commit would have



looked like a rewrite of the whole file. All the files I've sent now use CRLF, so each commit shows only that task's real changes:

Commit	Lines changed
Task 1	1
Task 2	11
Task 3	27
Task 4	8

I also made the missing `task2_game.py`. Put `task1_game.py` through `task4_game.py` into `Lab-4\code_by_task\`, replacing the old task3 and task4 copies, and your commit script will run end to end.

